INF10025 Data Management and Analytics

THEORY – Weeks 1 to 8

Complete revision notes for Test 3 (Distinction / High Distinction pathway)

Prepared for: Truong Ngoc Gia Hieu · Swinburne University of Technology · Semester 2 2026

Sources covered: every Lecture slide set of Weeks 1–8 (Modules), the Lecture Notes / Extra Note and the Week 1–3 Comprehensive Summary, the SQL Cheat Sheets, the Tutorial questions and answers (Weeks 1–8), Laboratory Tasks 1–5 and the Test 1 / Test 2 revision material.

Test 3 facts (Assessment Criteria): Test 3 is only for students aiming at Distinction or High Distinction (Tests 1 and 2 must be passed on the first attempt); it is marked out of 5 and the pass mark is 2.5. It takes place in the Week 9 tutorial. Lecture 8 states that outer joins, subqueries, unions, date functions and constraints are the “advanced” SQL expected from higher-grade students.

Part	Content
Week 1	Data categories, relational model, RDBMS, Access tables / keys, filters, queries, AND/OR criteria, wildcards
Week 2	Relationships, referential integrity, Cartesian product, totals queries, forms, indexes
Week 3	Parameter queries, action queries, crosstab, SQL behind Access, importing Excel / CSV / fixed-width / XML
Week 4	Power BI Desktop: connecting, relationships, visualisations, DAX columns, Access / Power BI functions
Week 5	ERD concepts, cardinality, identifiers, ERD → relational schema, Oracle iSQL Jr, CREATE / INSERT / SELECT / UPDATE / DELETE / ORDER BY / WHERE
Week 6	FK constraints, NOT NULL, participation, aliases, inner joins, AND / OR / NOT, UPPER / LOWER, BETWEEN / LIKE / IN / IS NULL / DISTINCT
Week 7	Composite keys, weak entities, aggregates, GROUP BY / HAVING, SQL in Access
Week 8	M:M expansion, 9-step ERD method, identifier testing, dates, subqueries, outer joins, UNION / INTERSECT / MINUS, literals, UNIQUE / CHECK
Appx	A Access vs Oracle · B Lab tasks · C Normalisation & transactions · D SQL cheat sheet · E Tutorial coverage

TIP – How to read this document

Boxes: **DEFINITION** (learn word-for-word), **EXAM TRAP** (typical mistakes), **TIP**, **NOTE**. SQL is shown in Oracle syntax unless stated; Access differences are flagged and collected in Appendix A. Slide typos are pointed out where they exist.

Week 1 – Fundamentals, DBMS Concepts and Tools

1.1 Understanding data (Lecture 1 Part 1)

Organisations need to store and retrieve usually very large amounts of data. Data can be divided into **three major categories**:

Category	Description	Examples / notes
Structured data	Follows a predefined, repetitive pattern or format; data types are specified for each piece of information. This is what Relational Database Management Systems (RDBMSs) use: Access, Oracle, SQL Server, MySQL.	University student data: Student ID (numeric), Student Name (alpha), Gender (alpha), DateOfBirth (date), HomeAddress (alpha), PhoneNo (numeric), NextOfKin (alpha). Every student is different but the university stores every student in the same format.
Semi-structured data	Information that does not match the requirements of a relational database, but is organised/arranged so it is easier to analyse.	XML documents and NoSQL databases (covered briefly in later weeks).
Unstructured data	Not organised in a pre-defined manner; the organisation does not know the format or content in advance; contents are unpredictable. One item may contain text, audio, video, links, images and data about many organisational functions.	Data sourced from social media and e-mail. Question raised: how do organisations store such data so it can be retrieved, collated and analysed? (future weeks)

1.2 The relational data model and the RDBMS

- RDBMSs are based on the **Relational Data Model**, developed by **Ted (E. F.) Codd in 1970**. Data is represented as **two-dimensional tables**.
- Each table has: a set of **uniquely named columns / attributes**; a list of **unnamed / unnumbered rows**; **the order of the rows is irrelevant** (if rows are shuffled the table keeps the same meaning).
- A **row** consists of a sequence of attributes – one cell for each attribute – and **only one value per cell** is allowed.
- A **relational database** is simply a collection of related tables. Example: Student table (students), Unit table (units of study), Enrolment table (units that a student has enrolled in).
- An **RDBMS** is a collection of programs that let developers/users store and retrieve data from relational databases and perform **CRUD** operations – **C**reate, **R**ead, **U**pdate, **D**elete (create a student, retrieve a student's details, update a student's details, delete a student from a table).

ID	EmpName	Branch	Salary
1001	Fred Blogs	Haw	89000
1004	Emma Jevs	City	125000
1006	Dave Rigg	Haw	65000

Lecture example: a table with uniquely named columns (attributes), unnamed and unnumbered rows, and one value per cell.

Setting up an RDBMS

- **Tables** follow a 2-dimensional structure; each row of data has a **Primary Key** value; no duplicates (e.g. Student ID).
- **Constraints** can be added to validate data: Student ID is the correct length; Student type is PG or UG (post/undergraduate); the student is enrolled in a degree that actually exists.

Advantages of an RDBMS over a spreadsheet

- Constraints are usually easier to implement than in a spreadsheet – “lookups” and “rules” are easier.

- When searching, **only rows that match the criteria are retrieved**; a spreadsheet loads the entire sheet.
- The data exists in **one location** (e.g. a database server) – no multiple copies of "the same(?)" spreadsheet.
- Changes occur in a single place and **all users see the change immediately**; sheets may otherwise be consolidated later.
- Other benefits (covered over the next weeks): **concurrency**, **eliminating data redundancy**, **relationships with other tables**.

RDBMS and database servers

- A **database server** is a computer networked to other computers; it stores databases; users on the network can access the data. There is only **one copy** of the data (excluding backups).
- RDBMSs allow **multiple users** to update data at the same time – many people can check the price of product 20; many may enrol in INF10002 simultaneously; many users may purchase tickets for a flight at the same time.
- Why not a word processor or spreadsheet? Telstra (all customers) or ANZ (all deposits and withdrawals of all customers – thousands/millions/billions of rows). Retrieving all deposits made by customer 1234 in the past 24 months from a spreadsheet would suffer from: **size of sheets, load time, lack of computer memory, unnecessary loading of other customers' transactions**.

Two ways to interact with a database

	GUI (e.g. Microsoft Access)	SQL interface
How	Mouse clicks, pull-down menus, drag-and-drop, minimal typing	Every action is a very strictly-written SQL command
Pros	Easy to use (after initial training)	Useful for programmers/developers who want to **automate** common tasks; typing a command can be quicker than many clicks
Cons	Complex actions are slow (many menu options and filters) and **cannot easily be saved** – they must be replicated again and again	Relatively difficult to learn; requires typing skills; strict syntax

Many RDBMSs offer a combination of GUI and SQL interfaces. Lecture example of SQL:

```
SELECT    movieno, title, relyear
FROM      movie
WHERE     (relyear >= 2005 And relyear <= 2009)
AND       title IS LIKE "%Harry Potter%"
ORDER BY  relyear
```

> **NOTE – slide as printed**
> The slide writes `title IS LIKE` – the correct SQL is simply `title LIKE '%Harry Potter%'` in Oracle (Access uses `Like "*Harry Potter*"`).

1.3 MS Access – tables, records, fields, primary keys (Lecture 1 Part 2)

- Access is a **complete RDBMS**, mainly used by individuals or small organisations; a good learning tool for students; **requires MS Windows** (Mac users: VirtualBox, Apps on Demand – Citrix); installed on all on-campus PCs.
- Access users typically do not create a database server and usually do not share data with other users; Swinburne labs do not easily allow sharing – so in the labs you create **single-user databases**. Multi-user environments are met in other units.
- **Design view** of a table shows details of all fields. Each field has a **data type** (what type of data may be stored). This semester: **Short Text, Number, Currency, Date**. The **key symbol** marks the Primary Key (e.g. movies are uniquely identified by MOVIENO).

- **Datasheet view** shows the rows. Not all rows can be loaded at once (millions of rows). **Dangerous:** if you change a cell and move to a new row, the data is changed **permanently** – there is no "Do you want to save changes?" question.

Relational model	MS Access	Oracle / SQL
Relation	Table	Table
Attribute	Field	Column
Tuple / Row	Record	Row

Most databases and DB developers say **Row** and **Column**; Access says **Record** and **Field**. The terms are used interchangeably. Example: the movie table contains 312 records; each record holds data about one movie and is made of many fields.

- Field Properties / Property Sheet appear in Design view – "don't be overwhelmed": most are ignored this semester.
- **Primary Key**: every table must have one. Optional in Access, but any table based on Codd's model must have it. It is a field whose value is a **unique identifier** for each record; **no duplicates**; the data should be **static** (should never need to change). Select the field and click the key icon.
- **AutoNumber**: if you forget to nominate a primary key Access offers to create one with the AutoNumber data type; Access simply allocates sequential numbers, so the user never types a new student number. Very useful, but **avoid AutoNumber in the first weeks of semester**.

> **DEFINITION – Extra Note – the four main Access objects**
> **Tables** – store data; **Queries** – organise data; **Forms** – display data on screen; **Reports** – print out data.

1.4 Filters and queries (Lecture 1 Part 3)

- **Datasheet filters**: click the drop-down on a field heading, tick boxes to choose values, "Select All" toggles all. *Text Filters*, numeric filters and date filters (huge selection) offer further options.
- **Filters are for one-off searches** and filter settings **cannot be saved** – you must re-create them next time. **Queries** are most useful when a search is repeated: a filter can be saved as a query, a query can be re-run many times, a query behaves just like a table, and a new query can use an existing query as its source.
- An Access query is literally a **question you ask about your data**, e.g. "What is the name and rating code of all movies released in 2015?". To build one think about: which table(s) store the data; which fields to display; what criteria a record must match; what sequence the result should appear in.

Worked example ("title and rating code of all movies released in 2015"): table = Movie; fields = Title, RatingCode; criteria = Relyear = 2015 (add Relyear to the grid but untick **Show**); sort = alphabetical by title. Click **Run (!)** to execute – the datasheet shows the matching rows.

- **Design Grid rows:** Field, Table, Sort, **Show** (tick box = column visible in results), Criteria, or. Add fields by drag-and-drop or double click.
- A query is saved inside the database (e.g. "Movie Title and Ratings of 2015") and re-run by double-clicking it. **Reusing queries**: create a new query via Create / Query Design, choose the saved query on the Queries tab instead of a table (only its columns are available), e.g. add RatingCode = "PG" to show only PG movies of 2015.
- Query reference sites listed in the lecture: The Access Bible 2016 (Swinburne library), support.office.com "Examples of query criteria", "Use the OR criteria…", gcflearnfree.org Access 2013 query-criteria quick reference, fontstuff.com acctut06.

Multiple criteria – AND / OR logic in the design grid

Layout in the grid	Meaning	Example
Several values in the **same field on different rows** (Criteria, or)	A record needs to satisfy only **ONE** of them (OR)	relyear = 2013 OR relyear = 2016
Values in **several fields on the same Criteria row**	A record must satisfy **ALL** of them (AND) – reading across the query means AND	relyear = 2013 AND ratingcode = “M”
Multiple criteria rows with values in several fields	Evaluate each row separately: select the record when **all of Criteria A** is satisfied **OR all of Criteria B** is satisfied	(A) 2013 and M, or (B) 2016 and PG
OR keyword in a single cell	Alternative to using multiple rows	2013 Or 2016
OR keywords in two fields	(relyear = 2013 OR relyear = 2016) AND (ratingcode = “PG” OR ratingcode = “M”) → 2013&PG;, 2013&M;, 2016&PG;, 2016&M;	relyear: 2013 Or 2016; ratingcode: “PG” Or “M”
AND keyword in a single cell	Most useful for numeric ranges	relyear: >=2011 And <=2015 (between 2011 and 2015)

EXAM TRAP – Be careful with AND
relyear = 2013 AND relyear = 2016 in the same field is **impossible** – no row can have two values – the result is empty (“No rows match this criteria”). Think about what you are really searching for.

Text criteria, wildcards and field names

- Text values in the Criteria cell are surrounded by **double quotes**. **Access is NOT case sensitive**: “The”, “THE” and “the” are the same. The opposite is true in other databases – in **Oracle** “THE” differs from “The” and “the”.
- Wildcards must be used with the **Like** keyword (pattern matching). Mind the space: `Like "the *"` (with a space) only finds titles starting with the word “the”; without the space you also get “Thelma and Louise”.

Wildcard (Access)	Description	Example	Meaning
*	any string of any length (0 or more characters)	Like "the *"	Title must begin with “the ”
		Like "*day*"	Title must contain “day”
?	exactly one single character	Like "the ????"	Title begins “the ” followed by exactly 4 characters
# (comprehensive summary)	one single digit 0–9	Like "1#3"	103, 153, 183 …
[charlist] / [!charlist]	any character in / not in the list	Like "[M-N]*" / Like "[!A-C]*"	begins with M or N / does not begin with A, B, C

TIP – Field names
Avoid spaces; use letters A–Z, a–z and digits 0–9. Square brackets such as `[movieno]` `[title]` `[relyear]` exist because Access allows spaces in field names (please avoid them – “leads to headaches”); [ ] is the only way to tell Access where a field name starts and ends.

Field properties (Weeks 1–3 comprehensive summary)

Property	Meaning / use	Example
Field Size	Limits the number of characters (Short Text) or the capacity of a number type (Integer, Long Integer)	Field Size = 50 for Last Name
Format	How data is displayed on screen	Medium Date (21-Sep-26), Currency ($)
Input Mask	Template that forces the entry structure	(0#) #### #### for a phone number
Default Value	Value filled in automatically for a new record	="VIC" for State, =Date() for creation date
Validation Rule	Condition the entered data must satisfy	>=0 And <=100 for marks
Validation Text	Error message shown when the rule is broken	“Mark must be between 0 and 100!”
Required	If Yes the field cannot be left Null	Required = Yes for Student No.
Indexed	Builds an index to speed up searching and sorting	Yes (No Duplicates) for the Primary Key

How the lecture tasks are used

- Tutorial questions are reviewed after the lecture and discussed in the tutorial (tutorials usually discuss topics from the previous week’s lecture). Topics discussed may appear in **tasks and tests**.
- Submission tasks are completed in your own time (check deadlines), need MS Access for the first weeks, and are submitted via Canvas.

Week 2 – Relationships, Indexes and Queries

2.1 Multiple tables and relationships (Lecture 2 Part 1)

Adding a record by typing into the Movie datasheet is **not end-user friendly** (customers should not use a datasheet), would expose personal details of other employees in an employee table, and makes it easy to delete one or many rows by mistake. The Movie table has a field **RatingCode**: what is a "PG" movie? RatingName "PARENTAL GUIDANCE", Description "Parental guidance recommended for persons under 15 years". What is a "MA" movie? "MATURE ACCOMPANIED (15+)".

Too much in one table → separate table

If ratingName and ratingDescription were added to MOVIE, the same text would be repeated on every PG or MA movie row – **redundant data** that wastes space and can become inconsistent (the "problems" question on the slide). **Solution: create a separate table for Ratings**.

RATINGCODE (PK)	RATINGNAME	DESCRIPTION
G	General	Suitable for all ages
PG	Parental Guidance	Recommended for mature audiences 15 years and over
M	Mature (15+)	Persons under 15 years must be accompanied by a mature adult
MA	Mature Accompanied (15+)	Parental guidance recommended for persons under 15 years
R	Restricted (18+)	Restricted to mature adults 18 years and over

- With more than one table open the **Relationships** window, add the tables, and **drag the field RATINGCODE in the movie table onto RATINGCODE in the Rating table**.
- This creates a **Many-to-One relationship**: **many FKs refer to one PK**. **movie.ratingcode** is a **Foreign Key (FK)**; each FK value in Movie references a Primary Key value in Rating.
- A query using both tables shows the relationship; it matches every FK value in Movie with a PK value in Rating.
- **Referential integrity** ("Enforce Referential Integrity") is a constraint the RDBMS performs: only values in movie.ratingcode that match an existing value in the rating table are allowed. Entering ratingcode "X" causes a **referential integrity error**.

EXAM TRAP – Unrelated tables and Cartesian products
If no relationship exists the RDBMS does not know what the tables have in common. A query using both returns a **Cartesian product**: every movie row matches every rating row. 318 movies × 5 ratings = **1590 rows** – utterly useless, can be very slow – **AVOID**.

- MOVIE also has Colour_Code → create table **ColourType** and add it to the Relationships manager → **two Many-to-One relationships** from MOVIE.
- ACTOR (actorNo, firstname, surname, gender) and MOVIE: to record which actors appear in which movie create a table **CASTING**. Each record represents **one actor in one movie** (rows like 51329/82693, 72129/82693, 72129/70160). CastId is just an auto-generated number to give each row a unique identity (primary key).
- The relationship diagram "looks complicated" but is simply a collection of multiple **Many-to-One** relationships; each relationship has a FK that references a PK.

DEFINITION – Three relationship types (comprehensive summary)
1:M – one record in table A (PK) relates to many in table B (FK), the most common (RATING 1 – M MOVIE). **M:N** – cannot be joined directly; split into two 1:M relationships through a junction table (ACTOR – CASTING – MOVIE). **1:1** – one record in A to exactly one in B, used for security or to separate sensitive data (EMPLOYEE – EMPLOYEE_SALARY).

2.2 Queries with totals – aggregation (Lecture 2 Part 2)

Users often want totals: How many employees? How many students are enrolled in INF10025? How many staff of each gender? In the database world this is called **aggregation**; typical aggregate functions are **Count, Sum, Min, Max, Average**.

- Requirement example: “Count how many movies belong to each rating code” – column 1 = Rating Code, column 2 = Count (G 24, PG 45, M 121, MA 83, R 5).
- In the Query Design ribbon click **Totals (Σ)**: a **Total** row is added to the grid. Default operation is **Group By** – all equal RatingCodes form a group (G is a group, PG is a group …). Alone this only lists the ratingcodes that appear in Movie.
- Add RatingCode a **second time** and change its Total to **Count** → count per ratingcode.
- **Changing the column heading**: the default *CountOfRATINGCODE* is not professional; put the new heading followed by a colon in front of the field name, e.g. `Total: RATINGCODE` (applies to any column heading in any query).
- Group by **relyear** and count → movies per year. Two grouping columns (relyear, ratingcode) + count → e.g. 2016 has 11 movies: 7 M, 2 PG, 1 G, 1 MA.
- **AVG** example: average TMDB score for each year. Use **Round(value, decimal places)** to round, e.g. `Round(Avg([TMDB_SCORE]),2)`; Access names the new column **Expr1** – replace the prefix with any heading you want.
- A **criteria on an aggregate** applies to the totals: e.g. only show years whose average TMDB score is **>= 7.4**.

2.3 Forms (Lecture 2 Part 3)

- Access lets developers quickly create input forms. A simple form shows **one record at a time** – easy for the end-user to focus on one item, difficult to delete or change many records by mistake.
- Create with the **Form Wizard**: choose which columns appear (some calculated or later-populated columns need not appear); choose the layout (**Columnar** lets you place and resize all fields); give the form a name, e.g. “Movie Form”.
- A more professional system adds menus/buttons for CRUD operations, colour, text, images, hints – but forms with buttons, menus, hints, validations, user-defined error messages and sub-forms are **not “simple to create”**.

2.4 Indexes (Lecture 2 Part 3)

When a Primary Key is created the field property panel shows **Indexed**. **What is an index and how can it help?**

- Without an index Access must read each record sequentially to find an employee id (100,000 records: “a second or two”; 10 million rows: longer). The more rows, the slower the sequential search. With a sorted list of ids (numeric sequence) the search is much easier.
- An index stores **the index value (e.g. employee id)** and **the location of the record** in the database. Searching: find the id quickly in the index → get the location → retrieve the record.
- **Primary Key index**: whenever you create a PK Access automatically creates a PK index. Entering a PK value already used gives a **Duplicate Primary Key** error; Access detects this so quickly because the field is indexed.
- **Sequential record access**: read each record from the start until found; for 50,000 records the DBMS may evaluate 50,000 records, average search ≈ **25,000 disk I/Os** (disk I/O is the slowest computer activity). In reality a DBMS reads blocks (e.g. 2K) so many records come in one I/O.
- **Inserting into a simple index**: as each row is added the DBMS inserts the search key and row location (RowLoc) into the index file, kept **in key sequence** (e.g. insert employee 109 → row 1; insert 13 → row 2, index order 13, 109; insert 27 → row 3, index order 13, 27, 109).
- The index is much smaller than the table; searching it is quicker. A DBMS may use a **binary search**: start at the middle position; the value is either the current key, a higher key, or a lower key – at least half of the keys are eliminated; repeat with the remaining keys until found; then

fetch the row. Example: employee 289 found using the index took **2 disk I/Os** (one to read the index into RAM, one to read row 11).

- A binary search is only one method; real indexing is far more complex and most RDBMSs offer several methods (choice depends on the data). It is **not possible to display the contents of an index** (may be non-human-readable/compressed).
- The index does **not** contain a record number – the record number shown in a datasheet does not really exist (Fred Blogs in record 3 becomes record 1 if records 1 and 2 are deleted), which is one reason we use Primary Keys. The RDBMS uses a **physical address on disk** (RowLoc); in Oracle something like `AAAYUzAAEAABIjLAAJ`; Access makes it almost impossible to view.
- Printing all 100,000 employees in id sequence from an index means jumping from location to location (employee 4 at 97,491; employee 5 at 18; employee 7 at 43,292 …) – a lot of work; without an index it would be very, very slow. (For a table of a few thousand rows, index or sequential search is almost identical in speed.)

Advantages, creation and cost of indexes

- **Advantages**: quick way to locate rows; large portions of the index (or all of it) can be read into memory before searching; **fewer disk I/Os** → faster retrieval.
- **Index creation** is easy and automatic for Primary Keys. Some indexes can have **duplicate values**: Surname is a poor Primary Key (many people share a surname) but a good index because it is often searched or used for report order. With 100,000 rows and no index on surname an average search needs ≈ **50,000 reads**; with an index: one read to load the index into RAM and binary search, one read for the first matching row, subsequent reads for duplicate surnames.
- **Indexes speed up report generation**: reports using many tables with FK lookups and millions/billions of rows can take hours (a daily report taking >10 hours is next to useless); indexes on fields central to the report have cut times from many hours to a few minutes.
- **Cost – why not index every field?** Disk space is a minor consideration (cheap). **Update time is a major one**: when records are inserted, changed or deleted **every index must be updated** to stay in key sequence, so each change takes longer – too many indexes cause unacceptable delays (every unit enrolment, every census record).
- **Deletion / recreation**: for a once-a-week report needing many indexes it may make sense to create the indexes before the report, run it, then drop the indexes (often scheduled automatically).
- **Multi-level index**: very large indexes are split into smaller index segments with an “index of indexes” (LastKey → IndexSeg); to find employee 145 look in the top-level key list, go to the right segment, then to the row.
- **DBMS optimisers** decide the quickest and most efficient way to access data. Even if indexes exist, the optimiser decides whether to use them – for a very small table it may be quicker to load all rows than to load the index and then the row.

> **TIP – Index summary**
> Speeds up SELECT (searching, sorting, joins, reports) but slows INSERT / UPDATE / DELETE and uses space. PK index is automatic and has no duplicates; other indexes (e.g. Surname) may allow duplicates. Lab task Pass 1j: create an index on Surname (duplicates OK).

Week 3 – Queries, Action Queries and Importing / Exporting Data

3.1 Queries with parameters

- A query such as “movies with release year 2015” has the criteria typed in. Similar queries for 2014, 2013 … lead to **a long list of queries, clutter and confusion**.
- Alternative: a **Query Parameter** – a **temporary storage location that is given a name**, e.g. *Find Year*. In the criteria for year write = `[Find Year]`. When the query runs Access asks a question and then retrieves the correct records.
- Click the **Parameters** icon in the ribbon, create a **Parameter Name** (may contain spaces) and specify a **data type**: **ShortText, Date, Integer** (whole number, no decimal), **Decimal** (number with decimals). Add the parameter name to the Criteria of the field (e.g. RelYear) and run.
- Two queries with parameters can be combined: a new query uses “Movies of a Specified Rating and Year” as its source and adds a new parameter *Find Rating Code*; when executed **both parameters** are used.
- Parameter values can be used in expressions (Week 4): parameter [Enter Name] = “Fred” → = `"The parameter value is " & [Enter Name]` gives “The parameter value is Fred”; parameter [Enter Year] = “1998” → = `DateValue("15 May " & [Enter Year])` gives 15/5/1998.

3.2 Copying tables, Make Table and Append queries

- Copying a table is something you generally want to **avoid**: with a Student table copied, two tables store your id, name, address, phone. If you change your phone number which table is updated? You cannot guarantee both are – data can get **out of synch**. Storing the same data in two tables causes **redundant data** → **multiple versions of the truth** and **loss of data integrity**.
- However, to “play” with or test data you can copy a table (copy/paste in Access) and use the copy. (Lab task: STUCOPY9999.)
- **Make Table** query (Make Table icon, then Run) copies selected rows (e.g. the movies of the year 2000 into “Movies of 2012”) into a **new table**. Warning: this causes redundant data. Useful for testing.
- **Append query** is similar but instead of making a new table the selected records are **appended to an existing table**; the fields of the original table must match the fields selected in the Append Query. Make Table and Append should be used **sparingly (if at all)** – best for testing only.

3.3 Update and Delete queries (action queries)

- **Update query** changes the value of field(s) for **many records at one time** – e.g. set every employee’s HoursWorkedLastYear to zero; or raise HourlyRate by $2.50: the calculation includes the name of the field (`[HourlyRate]+2.5`).
- When run, Access gives a warning about impending changes (the warnings can be suppressed in Access Settings). Once you say Yes the records are changed.
- **There is no way to undo an update query – Ctrl+Z is useless.** It is possible to update every row of a 10-million-row table if the criteria is wrong and untested – catastrophic. **Take backups**, especially before such a query.
- **Delete query**: like an update query, rows are **permanently deleted**; no undo; backups are crucial.

3.4 Crosstab query

A Cross Tab query is similar to an Excel **Pivot Table**: it performs an aggregation (sum, count …) and groups the results by **two sets of values – one set down the side of the datasheet and the other across the top**.

3.5 SQL behind Access

- Almost all relational databases use **SQL (structured query language)** to manipulate objects and data. It is hidden in Access because Access uses a GUI and many non-computer-savvy people would have trouble understanding SQL.
- Each time you create a table or design a query **Access generates SQL behind the scenes**. Click the **SQL icon** (bottom right of the query design screen) to see the SQL statement.

3.6 Importing data

Data often exists before the database is built and must be entered into Access tables.

	Option 1 – Retype	Option 2 – Import using database tools
Result	Costly, prone to errors, slow	No cost (other than creating the process), fast, no typing errors, the same process is applied to every record, **conversion rules** can be applied, data can be converted from one data type to another

- Access can import from many sources: **Excel, Text file, XML, database to database** ...
- **Import vs Link** – Option 1 **Import**: make a copy of the original data stored in Access tables (most often used). Option 2 **Link**: only one version of the data; the source data is maintained by the original application; Access has a **read-only view**; used when Access works with data continually maintained by another application.

Importing from Excel

- Rules: data must resemble an Access datasheet; **uniform rows and columns**; may be in multiple worksheets; **every column must have a heading**.
- Wizard steps: choose the worksheet name; tick **“First row contains column headings”** (otherwise headings become field values and fields are named Field1, Field2, Field3 ...); specify which fields are **Indexed** (most important: the primary key); adjust each field’s **name and data type** (text often needs converting to a number); choose the **Primary Key** – let Access add an AutoNumber (new field) or pick an existing field.
- Option to **save the import steps** – useful for regular imports (every morning/week) or when an inexperienced person performs the process.

DEFINITION – Dates in Excel
Excel stores dates and times as a **serial number = number of days since 1 January 1900**; the fractional part stores the time. After import the values may look like numbers but Access interprets them correctly.

Importing from CSV / delimited / fixed-width files

- **CSV = Comma Separated Values**, also called a **delimited file** (values separated by a delimiter character, e.g. comma). Text may be wrapped in quotes: `101, "Dave", "Hillcrest", "2 Red Rd"`.
- In Access choose **Text File**, then **Delimited** (for CSV) or **Fixed Width**. Specify the **delimiter** (normally comma), that row 1 contains field names, and the **text qualifier** (the character surrounding text such as " or ').
- The **Advanced** button offers extra options, especially for dates – many formats exist: 25/09/2010, 09/25/2010, 2010/09/25, 12-Sep-2010, 25-09-2010, 09-25-2010, 2010-09-25, 25/09/10, 25/9/2010, 2010/9/25 ... (and many countries use a comma as the decimal point, €123,00).
- **Fixed-width / space-delimited files**: choose Fixed Width and place vertical lines where each field begins. Access often does not recognise formats (dates!) – click a field to change its Field Name and/or Data Type.

Importing XML

- Each piece of data is stored between **tag elements** – a start tag and an end tag; tags can have a **hierarchy**. Example: the <Customers> tag contains seven subordinate tags such as <CustId>, <Givenname>, <FamilyName> ...

> **NOTE – Why file formats matter**
> An organisation has many IT systems (accounting, stock/inventory, sales/ordering, student records) and exchanges data with others (list of orderable parts, VTAC/high-school student preferences, price list). Few systems are integrated, so data is exchanged via **CSV, delimited, Excel or XML files** – importing/exporting through Access is a very useful skill.

3.7 Join types and other Weeks 1–3 syntax (comprehensive summary)

Join type	SQL name	Result
Type 1 – Inner join (default)	INNER JOIN	Only records matching in **both** tables (intersection). 10 child records, only 5 match a parent → 5 rows.
Type 2 – Left join	LEFT JOIN / LEFT OUTER JOIN	**All** records of the table on the LEFT (before the word JOIN); unmatched right-hand columns are NULL.
Type 3 – Right join	RIGHT JOIN / RIGHT OUTER JOIN	**All** records of the table on the RIGHT (after JOIN); unmatched left-hand columns are NULL.

Syntax	Example
DELETE	`DELETE FROM MOVIE WHERE RatingCode='X';`
UPDATE ... SET	`UPDATE MOVIE SET Price = Price*1.1;`
SELECT ... FROM	Standard query
LIKE	`WHERE Title LIKE "*Star*";` (Access)
ROUND()	`Round(Avg([Score]), 2)`
Validation Rule	`>=0 AND <=100`

Week 4 – Business Intelligence Tools (Power BI)

4.1 Microsoft Power BI and importing data (Lecture 4 Part 1)

- Microsoft has several **Business Intelligence (BI)** products. **Power BI** is a suite of desktop and cloud-based web applications for analysing datasets; datasets can be local or cloud-based and connected to / imported from RDBMSs, spreadsheets, online data services etc. It can be added to the Office 365 suite.
- In the Power BI **web app** users analyse data with visualisations and dashboards. One dataset may produce many visualisations: best-performing customers, worst-performing salespeople, upward-trending products, complementary product sales.
- **Power BI Desktop ≠ Power BI web app.** Desktop is a **Windows application** (installed on Windows or a virtual Windows OS on Mac/Linux): importing data, manipulating data, writing formulas and creating measures are done in Desktop. Finished visualisations / reports / dashboards are **deployed to a cloud service** so the web app can be used. Desktop is used mostly by BI analysts and developers; the web app by end-users and BI analysts.
- **Known problem with Access:** both apps must be the same bitness – **32-bit Access needs 32-bit Power BI; 64-bit Access needs 64-bit Power BI**. All Swinburne labs have 64-bit Access and 64-bit Power BI Desktop (via Apps on Demand). If you cannot connect, **export tables to CSV files and import the CSV files** into Power BI.
- Help links in the lecture: "What version of Office am I using?", "Resolve issues importing Access databases in Power BI Desktop", Download Power BI Desktop. In the next weeks the focus is Desktop (datasets, reports, dashboards) – **not deploying to the cloud yet**; Power BI is updated every year so screens differ, the concepts do not.
- **Get Data** → Access (or choose "Get data from another source" → Access) → Connect → choose the **tables** (you may also connect to **queries**, called **views** in other RDBMSs). From Power BI's point of view there is **no difference between a table and a query** – both present a collection of records made of fields.

Left-hand icon	Window	Purpose
Report	Report window	Where you add visualisations
Data	Data window	View raw data, add columns, measures and formulas, remove unwanted columns, write **DAX** (Data Analysis Expressions – like Excel formulas plus functions for relational data and dynamic aggregation; covered later)
Relationship (Model)	Relationships window	View / create relationships between tables; relationships from Access are transferred (symbols differ slightly); you can add more (e.g. data from other sources)

TIP – Cross-filter direction
In the Relationships window double-click each relationship line (it shows a one-way arrow) → in **Cross filter direction** select **Both**. Repeat for all lines; each then shows two-way arrows.

4.2 Visualisations (Lecture 4 Part 2)

- Visualisations are created in the **Report window**: some resemble Excel charts / Power Pivot charts, many are unique to Power BI. Steps: (1) select a visualisation, (2) select fields to act as values.
- **Table** visualisation – very similar to an Access query; click field names to add columns.
- **Matrix** – similar to an Access **Crosstab query** / Excel PivotTable: choose a field for rows (BirthCountry), a field for columns (Gender) and a field for the aggregation (Count of ActorNo).
- A report can hold **multiple visualisations** at once (Actors by Birth Country, Actors by Gender).
- Format via the **Paintbrush** icon (some visuals have 10+ categories of settings). Settings apply to a single visualisation and cannot be saved separately (only the entire workspace as a **.PBIX** file); delete and recreate a visualisation and you must redo its settings.

- **Slicer** = a visual filter that applies to all other visualisations in the report. A report may have several slicers; paintbrush options: only one box ticked, multiple boxes, “Select All” box.
- **Enter Data → Create Table** lets you add a new table inside the Power BI dataset (example: MyLocations with ID and Suburb; every customer record has a location id 1–6). Add it to the relationship diagram: customer.locationcode references MyLocations.id.
- **Map visualisation** highlights geographic points; use any of: suburb, town, city, state, country name, postcode, longitude & latitude. Dots are sized by the **Size** property – use something meaningful like CountOfCustId or SumOfSaleQty (not ID). Power BI is a little North-America-centric, so ambiguous suburb/town names need more specific information.
- **New column** (New Measure icon → New Column): in the formula bar replace “Column =” with the new field name, then =, then the calculation, e.g. `FullAddress = [suburb] & ", Victoria, Australia"` – the ampersand & **concatenates** strings. Use the new field in the Map for better results.
- You can set a field’s **data category** (e.g. City, State) so locations are confined to the specified country/state.
- **Tree Map** – a rectangle per data value; the bigger the rectangle the bigger the value (e.g. count of castings per actor; many rectangles are tiny). Most visuals have a **Visual Filter** setting, e.g. minimum Count of Castings = 10.

4.3 Access and Power BI functions (Lecture 4 Part 3)

Many functions have identical syntax in Access and Power BI (see techonthenet.com/access/functions). The ampersand joins strings; numbers are converted automatically:

```
= [movieno] & " " & [title]
= [movieno] & " " & [title] & " " & [relyear]
```

Function	Meaning	Example with Title = "The Matrix"
Left(string, n)	first n characters	Left([title],3) → "The"
Right(string, n)	last n characters	Right([title],4) → "trix"
Mid(string, start, n)	n characters from position start	Mid([title],5,3) → "Mat"
Ucase() / Lcase() (Access)	upper / lower case – non A–Z characters unchanged	Ucase([title]) → "THE MATRIX"
Upper() / Lower() (Power BI)	same, Power BI spelling	upper([title]) → "THE MATRIX"; lower([title]) → "the matrix"
MOD(number, divisor)	remainder after division	MOD([relyear],4)
Date() (Access) / Today() (Power BI)	current date	–
Now()	current date and time in both	–
DateValue(string)	date from a string	DateValue("1 Jan 2015"), DateValue("1 January, 2015"), DateValue("1 Jan 15") → 1/1/2015

EXAM TRAP – Not zero-based
Access and Power BI string positions start at **1**, not 0 (unlike many programming languages).

Week 5 – SQL and Modelling (Part 1): ERDs, Relational Schema, Basic SQL

5.1 Entity Relationship Models and Diagrams (Lecture 5 Part 1)

Creating a table is easy; creating multiple tables is just as easy. **Deciding what data is stored in each table is difficult**. Entity Relationship Models and Diagrams assist the design.

	Definition
Entity Relationship Model (ERM)	A **logical representation of data required by an organisation**: uses entities to represent people, objects, events …; identifies relationships between entities; based around the **business rules** of the organisation.
Entity Relationship Diagram (ERD)	A **graphical representation of the ERM**. An ER Model / ER Diagram does **not store any data** – it is a **plan**. It expresses the structure of information used by an organisation, and assists database design.

EXAM TRAP – ERD vs Access Relationship Diagram

An ERD uses **natural keys** – no surrogate keys (only real-world keys; discussed later). An Access Relationship Diagram matches the **existing database tables**; if surrogate keys are used in a table they appear in it. **An Access Relationship Diagram is not an ERD**: an ERD exists **before** the database is built; the Access diagram occurs **after**.

Three components of an ERD

- **Entities** – e.g. Subject and Lecturer (written in UPPERCASE, singular nouns specific to the business).
- **Attributes** – six in the example: SubjectCode, Title, CreditPoints, LecId, LecName, Age.
- **Relationships** – a line between entities, e.g. Subject “is convened by” Lecturer.
- When building an ERD **always consider example data** (that is why we build a database): Subject data INF10002 “Database, Analysis & Design” 12.5; INF10003 “Intro to Business Info Systems” 12.5; Lecturer data 207 John Smith 37; 119 Jane Pitt 26.

Business rules

- Examples: a student must only be enrolled in one course at any time; a student may enrol in many subjects at one time; a subject must only have one convenor; an employee must only have one tax file number. Some rules are common, others unique; **very few organisations have identical business rules**. Your ERD and database design must meet the organisation’s business rules.
- “Documenting rules and policies of an organization that govern data is exactly what data modeling is all about.” – Hoffer, Prescott, McFadden, *Modern Database Management*. There is an industry built around obtaining business rules; data modelling cannot easily be “off-shored”.
- **Why discovering business rules is hard:** no single person knows all of them; individuals cannot or will not tell you every rule (some tell you rules that do not exist – fear, distrust, changing jobs); lack of documentation; rules change (business needs, regulations, competition, technology) even during modelling. Rule gathering is covered in other subjects (RAM). **In DAD you are given all the business rules.**

Cardinality constraints

Cardinality specifies **how many instances of one entity are related to instances of another entity**. The answer is always **ONE or MANY**. At Swinburne: how many lecturers convene a single subject? ONE. How many subjects can a lecturer convene? MANY. On the relationship line **ONE = a single line; MANY = a crow’s foot**.

- **Determining cardinality – part 1**: take the relationship “convened by”; consider the LEFT end; take just ONE instance of Subject (INF10002): how many lecturers convene it? ONE → draw the

cardinality symbol at the RIGHT end of the relationship. Read left→right starting with one instance of Subject: “One Subject is convened by ONE Lecturer”.

- **Part 2**: consider the RIGHT end; ONE instance of Lecturer (Fred Smith): how many subjects does he convene? MANY → draw the symbol at the LEFT end. Read right→left starting with one instance of Lecturer: “One Lecturer convenes MANY Subjects”.
- **Summary:** ONE Subject is convened by ONE Lecturer; ONE Lecturer convenes MANY Subjects. **Always begin each sentence with the word ONE (never “many”).** The relationship name “convened by” only reads left to right – rephrase it when reading right to left.

Entity instances and identifiers

- An **entity instance** is one set of values for the attributes of an entity, e.g. LecId 207, LecName John Smith, Age 37 (another instance: 119, Jane Pitt, 26).
- An **identifier** is an attribute (or attributes) that uniquely identifies an entity instance – same concept as a **Primary Key**. Rules: **every entity must have an identifier; every instance must have a unique identifier; no duplicates; the value cannot be empty / null**. Identifiers are **underlined** on the ERD.

5.2 ERD to Relational Model (Lecture 5 Part 2)

- **Relational Data Model (RDM)**, E. F. Codd 1970: data in 2-D tables; each table is a **relation** with a set of uniquely named columns called **attributes**, a list of unnamed/unnumbered **rows (tuples)**, order irrelevant (shuffling keeps meaning), a row = sequence of cells, one cell per attribute, one value per cell.
- **Conversion rules:** the name of each entity becomes the name of the relation; each attribute becomes an attribute of the relation; **underline the Primary Key**. Relation names are normally UPPERCASE; attribute names lowercase or Capitalised: LECTURER (LecId, LecName, Age) is a **Relational Schema**. Written with the key stated: LECTURER(LecId, LecName, Age) Primary Key (LecId).
- Two unrelated entities give two relations with nothing associating a subject with a lecturer.
- **Converting the M:1 relationship:** every M:1 relationship generates a **single Foreign Key** (several columns if the key is composite). The FK column is added to the table at the **MANY end**; its name is usually spelt the same as the PK it refers to. **An attribute named LecId is NOT added to the Subject entity in the ERD** (a single attribute name must appear only once in an entire ERD) – the FK exists only in the relational schema.

```
LECTURER (LecId, LecName, Age)
SUBJECT  (SubjectCode, Title, CreditPoints, LecId)
Foreign Key (LecId) References LECTURER
```

- **Relation vs Relationship**: LECTURER and SUBJECT are relations; the line between them is a relationship (confusingly similar terms).
- Check question from the slides: after converting an ERD with DEPT and WORKER, count the columns of each table (including the FK).

Relational model	MS Access	Oracle
Relation	Table	Table
Attribute	Field	Column
Tuple or Row	Record	Row

NULL values

- A **Null** is a special value assigned to an attribute when a value is **unknown or inapplicable**. It is **not a space and not zero**; it has the value NULL.
- “Is Karen younger than John?” – Unknown (neither TRUE nor FALSE). If Karen Fry (LecId 771) has no Age, “select all lecturers older than lecturer 207” does **not** list Karen. Be careful with query results where NULLs may be found.

5.3 Oracle iSQL*Plus Junior and SQL (Lecture 5 Parts 2–3)

- For a few weeks you use an **Oracle database** through **iSQL Jr** – Swinburne's web interface (https://feenix-isqljr.swin.edu.au/), no software to install, usable anywhere; all data is stored on a Swinburne server.
- Login: **s** prefix + student ID (e.g. s123456789); password = date of birth in **ddmmyy** (e.g. 251299); database name **DAD**.
- Your Oracle account is private (not shared); it holds your tables; once a table is created and loaded it is permanent (until end of semester or until you drop it); tables are restricted in number and capacity.
- Type SQL in the text box and click **Execute**; results appear in the result set below.
- **Lab 1**: download `CreateMovie01.TXT` from Canvas, open in Notepad/Notepad++, select all and copy, paste into iSQL Jr and Execute. Ignore the error caused by the DROP TABLE statement: creating an existing table gives an error (hence DROP TABLE at the top), but dropping a table that does not exist also gives an error.
- Data types: **VARCHAR(n)** for text (requires a maximum length); **NUMBER(p,s)** for numeric data (optional max length and decimal places).

```
CREATE TABLE MOVIE (
  MOVIENO      NUMBER(6, 0) PRIMARY KEY
, TITLE        VARCHAR(100)
, RELYEAR      NUMBER(4)
, RUNTIME      NUMBER(4)
, RATINGCODE   VARCHAR(2)
, COLOUR_CODE  VARCHAR(1)
, TMDB_SCORE   NUMBER(3,1)
, TMDB_VOTES   NUMBER
, TMDB_ID      VARCHAR(12)
);
INSERT INTO MOVIE (MOVIENO, TITLE, RELYEAR, RUNTIME, RATINGCODE, COLOUR_CODE, TMDB_SCORE, TMDB_VOTES,
    TMDB_ID)
VALUES (620, 'Ghostbusters', 1984, 107, 'PG', 'C', 6.8, 570, 'tt0087332');
```

Why learn SQL (when Access has a GUI)?

- Programmers write SQL in code (C#, VB, PHP request student data: `"SELECT Id, Name FROM Student where Id = 1234"`). GUI software actually generates SQL (experts sometimes tweak it). DBAs/programmers use **DML** (data manipulation language). BI workers who know some SQL don't wait "3 days" for `SELECT * FROM STUDENT` to be coded. Power end-users use SQL when reports are lacking. Used again in INF20010 Database Systems.
- **Saving work**: SQL typed in iSQL Jr is **not saved** – copy and paste it into a text file on USB / network drive. **Notepad++** is free, more powerful than Notepad.exe, and highlights SQL (Language → S → SQL): keywords, comments, numbers, strings, functions. Other editors: Sublime, Brackets.

DDL and DML

Group	Meaning	Statements
DDL – Data Definition Language	maintains database structures / objects	CREATE TABLE, ALTER TABLE, DROP TABLE …
DML – Data Manipulation Language	works with the data in the database	INSERT, SELECT, UPDATE, DELETE …

CREATE TABLE

```
CREATE TABLE <table-name> (
  <column-name1> <data-type> [<max length>]
[ , <column-name2> <data-type> [<max length>] ] ...
[ , PRIMARY KEY (<column-name>) ]
[ , FOREIGN KEY (<column-name>) REFERENCES (<table-name>) ] ) ;
```

Oracle keywords in capitals; words in < > are user-defined; words in [ ] are optional. Short Text is replaced by **varchar()** and varchar needs a maximum length. **Commas and brackets are important** – errors if not used correctly.

```
CREATE TABLE Lecturer (
  LecId   number
, LecName varchar(50)
, Age     number
, PRIMARY KEY (LecId) ) ;
CREATE TABLE Subject (
  SubjectCode  varchar(10)
, Title        varchar(100)
, CreditPoints number
, LecId        number
, PRIMARY KEY (SubjectCode)
, FOREIGN KEY (LecId) REFERENCES Lecturer ) ;
```

INSERT

```
INSERT INTO <table-name> (<column-name1>, <column-name2>, ...)
VALUES (<value1>, <value2>, ...) ;
INSERT INTO Lecturer (LecId, LecName, Age) VALUES (207, 'John Smith', 37) ;
INSERT INTO Lecturer (LecId, LecName, Age) VALUES (974, 'Robert Tipping', Null) ;
```

- **Text data in single quotes** (not double quotes in Oracle); **numeric data without quotes** and with no punctuation (no commas, dollar signs). Unknown value → the keyword **Null** (no quotes – `'Null'` would be the text "Null").
- **Smart quotes**: MS Word/PowerPoint/OneNote replace ' by fancy ‘ ’; Oracle does not recognise them and gives **ORA-00911: invalid character**. Use Notepad/Notepad++; if forced to use Word, disable the AutoCorrect options.
- Course examples add Lecturers 207 John Smith 37, 345 Carol Kent 34 and Subjects (HIT1405 Intro to Web 12.5, 207), (HIT1407 EBIS 12.5, 345).

SELECT

```
SELECT <item>
FROM   <table name>
...
SELECT LecId, LecName, Age FROM Lecturer ;
```

Each column named in SELECT becomes a column of the result set, in the same sequence. * means all columns (`SELECT * FROM lecturer`); `SELECT age, lecname` returns Age first.

DELETE and UPDATE

```
-- Test with SELECT first, then change the first words:
SELECT * FROM Movie WHERE Run_time <= 100;
DELETE FROM Movie WHERE Run_time <= 100;

UPDATE <table-name>
SET <column-name> = <value> [ , ... ]
WHERE <condition>
UPDATE student SET Fees_Paid = 'Y' WHERE StuID = 1122334;
UPDATE employee SET salary = salary + 5000   -- slide shows a stray comma after 5000
WHERE UPPER(branch) = 'HAWTHORN' OR years_of_service >= 10;
```

EXAM TRAP – Warnings
Write a SELECT first, check the rows, then convert it to DELETE / UPDATE. **If you omit the WHERE clause ALL rows are deleted / updated.** Many DBMSs have no Undo ("Oops") feature. An UPDATE can set several columns separated by commas – but no comma after the last assignment.

ORDER BY

- Specifies the sequence of rows in the result set; use the column name or column number (position in the SELECT list); multiple columns allowed; each column **ASC** (default) or **DESC**.

```
SELECT LecName, Age FROM lecturer ORDER BY LecName;       -- ascending (default)
SELECT LecName, Age FROM lecturer ORDER BY LecName DESC;
SELECT LecName, Age FROM lecturer ORDER BY Age ASC;
SELECT LecId, LecName, Age FROM lecturer ORDER BY 3 DESC;  -- 3rd column of the select list
SELECT LecName, Age FROM Lecturer ORDER BY Age, LecName;   -- by age, then by name within each age
```

Coding rules and debugging

- Most errors come from **misspelling column names, table names, keywords, and leaving out commas**. Each of these fails: `SELECT LecId LecName Age FROM LECTURER` (no commas); `SELCT …`; `… FORM LECTURER`; `… FROM LECTURE` (wrong table); `LectName` (wrong column).
- **Not** typically causing errors: the **case** of keywords (SELECT = select), the **number of spaces**, splitting over **multiple lines**, and the **semicolon** (optional for a single statement but essential when combining multiple statements in a script).
- A SELECT can have extra clauses; they must be in the correct sequence; most are optional; each begins with a KEYWORD written in UPPER CASE for readability: SELECT, FROM, WHERE, ORDER BY.

WHERE – restricting rows

- `SELECT … FROM … [WHERE <search-condition>] [ORDER BY …]`. Each row is evaluated against the condition; if TRUE the row is in the result set. WHERE is often called a **Restriction** (reduces the number of rows).
- Comparison operators: **=** equal; **<>** not equal (most DBMSs also accept **!=**); **<**; **<=** (two keystrokes in that order); **>**; **>=**.
- Slide example on 7 lecturers (ages 21, 26, 26, 26, 34, 34, 37): Age = 34 → 2 rows; Age < 34 → 4; Age <> 21 → 6; Age >= 34 → 3; Age > 34 → 1; Age <= 50 → 7; Age < 20 → 0.
- **Numeric literals**: no quotes, no formatting; decimal points and leading minus allowed: `annual_budget < 50000` OK; `< 50,000` ✗; `< '50000'` ✗; `> 9.99` OK; `> -100000` OK.
- **String literals** must use quotes: Oracle, SQL Server, MySQL use **single quotes**; **MS Access uses double quotes**. `Gender = 'M'` OK; `Gender = "M"` ✗ (Oracle); `Gender = M` ✗.
- **Case sensitivity**: Oracle and MySQL treat upper/lower case as different ('smith' ≠ 'SMITH'); SQL Server and MS Access treat them as equal. With Oracle: `LecName = 'JANE PITT'` → 0 rows; `LecName = 'Jane Pitt'` → 1 row.

Listing existing tables, DESCRIBE, code style

```
SELECT table_name FROM user_tables;   -- or:  SELECT table_name FROM tabs;
DESCRIBE movie;                         -- column details of a table you have created
```

Code standards and style: put each clause on its own line, keywords in UPPER CASE, indent conditions. Real queries are often 20–30 lines; queries are read and modified by others; organisations have SQL style standards you must follow; “tutors hate debugging poorly styled queries”. Example of good style:

```
SELECT    rdate, temperature, rainfall
FROM      weather_readings
WHERE     temperature <= -6.5
OR        rainfall >= 1000
ORDER BY  rdate DESC
```

Week 6 – SQL and Modelling (Part 2): FK Constraints, Participation, Inner Joins, AND/OR/NOT

6.1 Foreign key constraints and relationships (Lecture 6 Part 1)

Recap: ERDs are based on business rules, describe the data an organisation wants to store and show relationships between entities (entity = real-world thing; attribute = property of an entity; relationship = between entities). Conversion steps: **Step 1** each entity becomes a relation; **Step 2** add attributes; **Step 3** identifiers become primary keys; **Step 4** the entity at the MANY end of each M:1 relationship gives a foreign key in its table. FK values must match PK values (or be null). The relational schema is the specification of the database; tables are built with SQL DDL with PK and FK constraints.

- In a FK–PK relationship the **Primary Key column is the Parent**; the **Foreign Key column is the Child**. The parent is at the **ONE** end of a M:1 relationship (a child has one parent); the child is at the **MANY** end (a parent may have many children).
- The **FK constraint** forces the RDBMS to validate the value entered in the FK column; an error results if the FK value does not match a primary key value in the related table.
- `ORA-02291: integrity constraint (s1234567.SYS_C002969703) violated – parent key not found` – no parent value matched (e.g. LecId 222 when only 207, 119, 345 exist). A successful insert shows "OK. 1 rows" (a cryptic message meaning 1 row added).
- **Parent / child rules (Oracle default):** a parent cannot be deleted if a child exists (a child cannot become an **orphan**); a parent can be deleted only when its children no longer exist. The same rule applies in Access if "Enforce Referential Integrity" is ticked. Deleting Carol (345) who convenes INF11007 gives `ORA-02292: integrity constraint (…) violated – child record found`.
- **NULL foreign keys** are acceptable (e.g. a new subject INF21009 Advanced Facebook whose lecturer is not yet known: `VALUES ('INF21009','Advanced Facebook',12.5,Null)`).

NOT NULL constraint

- If the business rule says "Every subject MUST have a convener" a null LecId is unacceptable. Each column can have a **Not Null** constraint, usually in CREATE TABLE (can be added later). Violation → `ORA-01400: cannot insert NULL into (s1234567.SUBJECT.LECID)`.
- Many columns may be Not Null (e.g. Title and LecId in SUBJECT). **Primary keys do not need a NOT NULL clause – PK values cannot be null anyway.**

```
Create Table SUBJECT (
  SubjectCode  varchar(10),
  Title        varchar(100)  Not Null,
  CreditPoints Number,
  LecId        Number        Not Null,
  Primary Key (SubjectCode),
  Foreign Key (LecId) References LECTURER ) ;
```

Participation constraints in ERDs

- Similar notion to Not Null constraints; drawn on the relationship line next to an entity. A **single line |** = **MUST** (mandatory): a subject must be convened by a lecturer. A **circle O** = **MAY** (optional): a subject may be convened by a lecturer. Read: "One Subject **must** be convened by One Lecturer"; "One Subject **may** be convened by One Lecturer".
- **Cardinality** is vital – it determines the number of tables. **Participation** is less important: it does not change the number of tables or columns; it only specifies whether **Not Null constraints** are required.

- Generally the **MANY end has optional participation** (a parent MAY have many children) – always draw optional at the many end. **Mandatory at both ends causes dilemmas**: "a Child must have a Parent and a Parent must have a Child" – a parent cannot be added without a child and vice versa. Developers sometimes switch such constraints off while loading initial data – not recommended day to day.

Calculations, headings, qualification and aliases in SELECT

```
SELECT Name, Test1, Test2, Test1+Test2 FROM StudentResults ORDER BY 4 DESC;   -- sort by the 4th column
SELECT Name, Test1, Test2, Test1+Test2 AS "Total  Score" FROM StudentResults;
SELECT Name "Student   Name", Test1, Test2, Test1+Test2 "Total  Score" FROM StudentResults ORDER BY
    Name;
SELECT Name StudentName, Test1+Test2 Total FROM StudentResults;
```

- The SELECT clause may contain column names, **expressions or calculations**. **Alternative column headings** (only used in the result set): AS keyword (optional); new heading in **DOUBLE QUOTES** (optional if one word).
- **Table name qualification**: prefix columns with the table name, `SELECT lecturer.LecId, lecturer.LecName FROM lecturer` – useful when two or more tables are involved.
- **Table aliases**: temporary names that only apply to the current statement. Once an alias is used you **cannot** refer to the original table name in that statement. `FROM lecturer L WHERE L.age >= 50`; `FROM wr207_V_X5 budget WHERE budget.year = 2011` (budget is the alias for a long table name).

6.2 Inner joins (Lecture 6 Part 2)

- A query often needs data from two tables related by FK–PK, e.g. "List all subject codes and their matching convenor names".
- **Cartesian product trap**: `SELECT SubjectCode, LecName FROM Subject, Lecturer` matches every row of the first table with every row of the second (4 subjects × 3 lecturers = 12 rows) – usually a mistake. It happens because SELECT **ignores all referential integrity constraints**; SQL has to allow queries on non-related tables (even from different databases). Solution: **you must specify how two tables are related in every SELECT** – a **join**.
- **Inner join** returns only data that satisfies a FK–PK condition. The join condition is normally `<foreign-key column> = <primary-key column>`.

```
SELECT    S.SubjectCode, L.LecName
FROM      Subject S
INNER JOIN Lecturer L
ON        S.LecId = L.LecId
```

- **Column prefixes:** if a column name is unique to one table the alias prefix is not required, but professionals prefix **all** column names with aliases (even for one table, even when unique) to future-proof queries – **always use alias prefixes**. An ambiguous column such as LecId (in both tables) must have the prefix.
- **Avoid old-style joins** (pre-2000, "dinosaur SQL developers"): `FROM Subject S, Lecturer L WHERE S.LecId = L.LecId`. They clutter the WHERE clause, may require extra brackets and are a source of bugs: `WHERE S.LecId = L.LecId AND (L.Age > 50 OR L.Title = 'Professor')` differs from `… AND L.Age > 50 OR L.Title = 'Professor'`.
- **Multiple relationships:** add OFFICE: "One Lecturer may be located in ONE office; one office may have MANY lecturers". Each M:1 relationship always generates a FK, always at the MANY end. Schema: OFFICE(OfficeNo, YearPainted, SqMeters); LECTURER(LecId, LecName, Age, OfficeNo) FK(OfficeNo) → OFFICE; SUBJECT(SubjectCode, Title, CreditPoints, LecId) FK(LecId) → LECTURER.
- Questions: "How large is the office of the convenor of INF11002?", "When was the office of the convenor of Media Thesis A last painted?" → need 3 tables / 2 inner joins:

```
SELECT    S.SubjectCode, L.LecName, O.SqMeters
FROM      Subject S
INNER JOIN Lecturer L ON S.LecId = L.LecId
INNER JOIN Office   O ON L.OfficeNo = O.OfficeNo
```

```
ORDER BY  L.LecName, S.SubjectCode
```

6.3 AND and OR operators (Lecture 6 Part 3)

- Requirement "list all the Smiths and Nguyens": `WHERE Surname = 'Smith' OR Surname = 'Nguyen'` is correct; with AND the result is always empty. **Consider only a single row**: can that row's Surname be both 'Smith' AND 'Nguyen'? No – a column stores one value. Can it be Smith OR Nguyen? Yes.
- **Numeric ranges**: "salary in the range 20000 to 30000" is `salary > 20000 AND salary < 30000` (AND = the range common to both conditions). `salary > 20000 OR salary < 30000` covers every value (the range covered by either line).
- The WHERE condition evaluates to TRUE or FALSE for each row; TRUE → row in the result set.

AND	Cond1 TRUE	Cond1 FALSE
Cond2 TRUE	TRUE	FALSE
Cond2 FALSE	FALSE	FALSE

OR	Cond1 TRUE	Cond1 FALSE
Cond2 TRUE	TRUE	TRUE
Cond2 FALSE	TRUE	FALSE

- Several ANDs / ORs: deal with **one pair of conditions at a time**, left to right (e.g. age > 20 AND gender = 'M' AND branch = 'Haw' → evaluate the first pair, then combine with the third).
- **AND takes precedence over OR.** `WHERE branch = 'City' OR branch = 'Haw' AND gender = 'M'` means "any City person OR any male in Haw" – NOT "males in Haw or City". It is treated as `branch = 'City' OR (branch = 'Haw' AND gender = 'M')`.
- **Bracket-insertion method for mixed AND/OR:** (1) is there a mixture? If not, use the truth tables. (2) find an AND; (3) find the expression to its left; (4) insert an open bracket before it; (5) find the expression to the right; (6) insert a close bracket after it; repeat for every other AND; then treat each bracketed expression as one expression – all separated by ORs; if any is TRUE the row is selected.

Original	With brackets
gender = 'F' OR gender = 'M' AND age < 23	gender = 'F' OR (gender = 'M' AND age < 23)
age < 23 OR age > 30 AND gender = 'F'	age < 23 OR (age > 30 AND gender = 'F')
dept = 'A' OR qty >= 50 AND qty <= 100 OR branch = 'City'	dept = 'A' OR (qty >= 50 AND qty <= 100) OR branch = 'City'
dept = 'B' AND qty >= 100 AND qty <= 200 OR branch = 'Haw'	((dept = 'B' AND qty >= 100) AND qty <= 200) OR branch = 'Haw'
age = 21 AND gender = 'M' OR age = 23 AND gender = 'F' (slide asks: how many rows?)	(age=21 AND gender='M') OR (age=23 AND gender='F') → Dave (21,M) only; Jack is 23 but M → **1 row**
age = 19 OR branch='Haw' AND branch='Kew' OR age = 22 AND gender = 'M' OR gender = 'F' (slide: how many rows?)	age=19 OR (Haw AND Kew → never) OR (age=22 AND M) OR gender='F' → Rob (19), Sue (F) → **2 rows**

6.4 More operators (Lecture 6 Part 4)

- **Parentheses ( )** are evaluated first. `gender = 'F' OR Age < 20 AND branch = 'Haw'` differs from `(gender = 'F' OR Age < 20) AND branch = 'City'`. Parentheses can also be used purely for **readability** – easier for the author, for the programmer who changes it, and "for the person marking your assignment or exam".
- **NOT** negates the expression immediately to its right. `WHERE NOT (branch = 'City')` = all other branches; `WHERE NOT (branch = 'City' OR branch = 'Haw')` = all branches except City or Haw. A pair of conditions must be in parentheses; without them: A `Dept = 1 OR Dept = 2` (Amy, Bruce); B `NOT (Dept = 1 OR Dept = 2)` (Carol, Dave); C `NOT Dept = 1 OR Dept = 2` (Bruce, Carol, Dave) – three different results.

Order of precedence (highest first)
1. Relational operators (<, >, = etc.)

Order of precedence (highest first)
2. Brackets
3. NOT
4. AND
5. OR

UPPER() and LOWER()

`UPPER(<character expression>)` returns all characters in upper case; `LOWER()` in lower case. They **do NOT change the data in the database** – each simply returns a value for use in the query; characters outside A–Z/a–z (numbers, punctuation) are unaffected.

```
SELECT UPPER(empid), UPPER(fname), UPPER(sname), UPPER(address) FROM employee
SELECT * FROM employee WHERE UPPER(sname) = 'SMITH';      -- correct
-- WHERE UPPER(sname) = 'Smith'   is WRONG: UPPER(...) can never match lowercase letters
```

BETWEEN, LIKE, IN, IS NULL, DISTINCT

Operator	Syntax / meaning	Example
BETWEEN	[NOT] column BETWEEN value AND value – **inclusive**, equivalent to >= AND <=. Not widely used; no equivalent in most programming languages	WHERE age BETWEEN 30 AND 40 ≡ age >= 30 AND age <= 40
LIKE	Partial text match; must use the keyword LIKE (not =). **%** substitutes for 1 or more characters (zero or more in practice); several % allowed	WHERE UPPER(name) LIKE 'J%'
IN	[NOT] column IN (expr1 [,...]) – list of values, comma-separated; shorthand for OR	WHERE age IN (24,26,28,30) ≡ age = 24 OR age = 26 OR age = 28 OR age = 30
IS NULL	[NOT] column IS NULL / IS NOT NULL. **Never** = Null and never IS 'Null'	WHERE height IS NULL; WHERE name IS NOT NULL
DISTINCT / UNIQUE	removes duplicate rows from a result set; Oracle professionals prefer DISTINCT because UNIQUE has other meanings (unique constraints)	SELECT DISTINCT country FROM student

LIKE pattern slide (matching J%, %J, %J%, %JO%, %JON%, %J%-% after UPPER):

Name	'J%'	'%J'	'%J%'	'%JO%'	'%JON%'	'%J%-%'
Johnson	Y	N	Y	Y	N	N
Jonas	Y	N	Y	Y	Y	N
Jones-Taylor	Y	N	Y	Y	Y	Y
Taylor-Jones	N	N	Y	Y	Y	N
Major-Timms	N	N	Y	Y	N	Y
Pankaj	N	Y	Y	N	N	N

(Rows computed with UPPER(name) LIKE pattern. e.g. '%J%-%' needs a J followed later by a hyphen: Jones-Taylor and Major-Timms. Taylor-Jones has its J after the hyphen, so it fails.)

EXAM TRAP – Exam trap
`Select * from customer where Upper(location) = 'Haw'` returns 0 rows – the upper-cased value 'HAW' can never equal 'Haw'. `Upper(location) = '%H%'` also returns 0 (= does not use wildcards; use LIKE).

Week 7 – SQL and Modelling (Part 3): Composite Keys, Weak Entities, Aggregates, GROUP BY / HAVING

7.1 Natural keys, composite keys, weak entities (Lecture 7 Part 1)

- **Natural keys**: if an organisation has no Employee No we could use the employee name as the PK: EMPLOYEE(EmpName, PhoneNo, PayRate); TIMELOG(WeekNo, HoursWorked, EmpName) FK(EmpName) references EMPLOYEE.
- **Composite keys**: the PK is made from values of multiple columns: EMPLOYEE(GivenName, Surname, PhoneNo, PayRate). When the PK is composite, **any FK must be an identical match** for the PK: TIMELOG(WeekNo, HoursWorked, GivenName, Surname) FK(GivenName, Surname) references EMPLOYEE.

Strong and weak entities

- An entity is a person, place, object, event or concept. Two groups: **strong** and **weak**.
- **Strong entity**: "an entity type that exists independently of other entity types" (Hoffer, Prescott, McFadden); "an entity type that is not existence-dependent on some other entity type" (Connolly & Begg). It can be identified by **its own attributes** – an identifier can be chosen from its own attributes. Example: BUILDING (BuildingCode e.g. BA, Description "Business Arts").
- **Weak entity**: "an entity type that borrows part of, or all of its … identifier" (Mannino). Its identifier is partially or fully defined by identifiers from other related entities. Drawn with a **double outline**.
- Room data: RoomNo 403, RoomDesc Lecture Theatre, Capacity 85, Building BA. **Rule of ERDs: an attribute must only appear once in an ERD** – so BuildingCode is NOT repeated as an attribute of ROOM.
- Is RoomNo enough to identify a room? **NO** – Swinburne has many rooms numbered 403 (EN building, ATC building…). The identifier must be **BuildingCode + RoomNo** (e.g. BA 403) – a **composite identifier**, but BuildingCode is not an attribute of ROOM → it is **borrowed**.
- **How to show a borrowed identifier:** the relationship is part of the identifier, so **underline the relationship name**. The entity at the **MANY end** uses the identifier at the ONE end as part of its own. The underline is important.
- Reading the ERD: RoomNo underlined = part of the identifier; double outline = weak entity (part of identifier is borrowed); relationship "contains" underlined = the many end borrows the identifier of the one end → identifier of ROOM = BuildingCode + RoomNo. "One Room must be contained in one Building"; "One Building may contain many Rooms".

Converting the strong/weak ERD to a relational schema

- Create a table for each entity; make each attribute a column; for each strong entity choose an identifier as PRIMARY KEY; take the PK at the ONE end and make it a FK at the MANY end; **if the many-end entity uses a borrowed identifier, the FK becomes part of its PRIMARY KEY**. By convention list the borrowed attribute first.

```
BUILDING (BuildingCode, Description)
ROOM     (BuildingCode, RoomNo, Capacity)
           PK (BuildingCode, RoomNo)   FK (BuildingCode) references BUILDING
COMPUTER (CompId, Description, BuildingCode, RoomNo)
           PK (CompId)                 FK (BuildingCode, RoomNo) references ROOM  -- composite FK
```

SQL with composite keys – the ON clause needs **all** columns of the PK and FK:

```
SELECT *
FROM   ROOM R
INNER JOIN COMPUTER C
ON     C.BuildingCode = R.BuildingCode AND C.RoomNo = R.RoomNo
```

7.2 Intersection entities, SQL aggregate functions (Lecture 7 Part 2)

- Movie database: MOVIE(movieNo, title, relyear, runtime, rating), ACTOR(actorNo, firstname, surname, gender), CASTING(movieNo, actorNo). “Reverse-engineering” gives an ERD of many-to-one relationships: the identifier of CASTING (MovieActor) is **MovieNo + ActorNo**; CASTING is a **weak entity**; both relationship names are underlined; the entity has a double outline. “MovieActor” is not a natural real-world name – **use the term used by the industry/business you model** (CASTING).
- Completed movie ERD → relations: ACTOR (actorno, firstname, surname), CASTING (actorno, movieno), MOVIECOLOUR (colourcode, colourname), RATING (ratingcode, description), MOVIE (movieno, title, ratingcode, colourcode, relyear, runtime). **Which relations have FKs and how many?** MOVIE has 2 (ratingcode, colourcode); CASTING has 2 (actorno, movieno); ACTOR, RATING, MOVIECOLOUR have none.
- **Purpose of the weak entity**: CASTING allocates an instance of ACTOR to an instance of MOVIE (Ben Stiller is cast in Zoolander). Parallel examples (each a different business with strong entities that interact → weak entity): **ENROLMENT** enrols a STUDENT in a SUBJECT (identifier stuId + subjectcode; Jane Taylor is enrolled in Database Analysis…); **LOAN** records a loan of a BOOK to a PERSON (BookID + PersonID + Date; Lou James borrowed “How to Program” on 1 May and again on 13 Jun).
- Quiz from the slide: weak entity X with underlined relationships to W (and also relationships to Y, Z that are not underlined) → the identifier of X is XID + WID; schema X(WID, XID, XNAME, YID, ZID) PK(WID, XID) FK(WID)→W, FK(YID)→Y, FK(ZID)→Z. **Weak entities borrow the identifier from the entity at the other end of any relationship line that is underlined.**

SQL aggregate functions

- Aggregate functions (called “Totals” in Access) – Count, Sum, Min, Max, Avg – are applied over a number of rows at one time; WHERE and ORDER BY still work.
- **COUNT(*)** counts **all rows**. **COUNT(column)** counts rows where that column is **NOT NULL**. Example: COUNT(*) = 384 but COUNT(ratingcode) = 381 because three movies have a NULL ratingcode; if there are no NULLs both give the same result.
- With a WHERE clause only matching rows are counted: `SELECT COUNT(*) FROM movie WHERE relyear = 2004` → 19; … `WHERE relyear >= 2000 AND relyear <= 2009` → 117.
- SUM, AVG, MIN, MAX **must be given a column name**: `SELECT SUM(run_time) FROM movie; AVG(run_time); MIN(run_time); MAX(run_time); SELECT SUM(runtime) FROM movie WHERE UPPER(title) = 'HAMLET'; SELECT MIN(run_time) FROM movie WHERE year = 2005;`
- **ROUND(number, [decimal_places])**: if decimal places omitted → 0 decimals. `AVG(runtime)` = 121.5094339622641509433; `ROUND(AVG(runtime),2)` = 121.51; `ROUND(AVG(runtime)) AS "AVG_RUNTIME"` = 122.
- Items in the SELECT list are either **non-aggregate expressions** (column names TITLE, RUNTIME, RELYEAR, MOVIENO …) or **aggregate expressions** (AVG(RUNTIME), MIN(RELYEAR), SUM(RUNTIME), COUNT(*) …).

7.3 GROUP BY / HAVING and SQL in Access (Lecture 7 Part 3)

- **GROUP BY** must be used with an aggregate expression whenever you want a non-aggregate expression in the SELECT clause. Put the non-aggregate expression in **both** SELECT and GROUP BY; put the aggregate expression **only** in SELECT.

```
SELECT relyear, COUNT(*) FROM movie GROUP BY relyear;
SELECT rating_code, COUNT(*) FROM movie GROUP BY rating_code;
SELECT relyear, AVG(runtime) FROM movie GROUP BY relyear ORDER BY relyear DESC;
SELECT relyear, AVG(runtime) FROM movie GROUP BY relyear ORDER BY AVG(runtime) ASC;
SELECT relyear, COUNT(*) FROM movie GROUP BY relyear ORDER BY 2 DESC;       -- 2 = 2nd column of the
    result
SELECT relyear, rating_code, COUNT(*) FROM movie
```

```
GROUP BY relyear, rating_code ORDER BY relyear, rating_code;
```

- Without ORDER BY the result set appears in **random** year sequence – not user-friendly. Two non-aggregate expressions + one aggregate: e.g. in 2013 there were 4 'M' movies and 6 'PG' movies.
- **HAVING** keeps only the result rows whose aggregate meets a condition (e.g. only years with 5 or more movies). Usage: copy the aggregate expression from SELECT into HAVING: SELECT `relyear, COUNT(*) FROM movie GROUP BY relyear HAVING COUNT(*) >= 5`.
- **WHERE vs HAVING:** WHERE specifies which **rows of the table** are used to build the result set (evaluated before grouping); HAVING specifies which **rows of the result set** (groups) are kept or eliminated using aggregate expressions. Example: years 1990–93 by rating code → 10 rows; adding HAVING COUNT(*) > 1 removes groups with a single movie.

```
SELECT   relyear, rating_code, COUNT(*)
FROM     movie
WHERE    relyear >= 1990 AND relyear <= 1993
GROUP BY relyear, rating_code
HAVING   COUNT(*) > 1
ORDER BY 1, 2;
```

EXAM TRAP – Typical error message
`ORA-00937: not a single-group group function` – you mixed aggregate and non-aggregate expressions incorrectly. `SELECT relyear, COUNT(*) FROM movie;` ✗ vs `… GROUP BY relyear;` ✓. Tutorial 7: selecting BIRTHCOUNTRY while grouping by GENDER is invalid; `SELECT COUNT(*) FROM ACTOR GROUP BY BIRTHCOUNTRY` is legal but "useless" because you cannot tell which count belongs to which country.

SQL in MS Access

- Steps: Create a query by Design → add **no** table → click the **SQL** icon (bottom right) → type the SQL → click View to see the Query-by-Design grid / run it.
- After running, the SQL is often **reformatted with extra brackets**. With INNER JOINs in Access SQL you **must enclose the table names and join clauses in brackets** – always true when two inner joins are used, sometimes true with one.

```
SELECT E.enrolmentid, S.firstname, S.lastname, C.coursename, O.orgname
FROM ( ( (enrolment E
INNER JOIN student S      ON E.studentid = S.studentid )
INNER JOIN course C       ON E.courseid  = C.courseid )
INNER JOIN organization O ON S.orgid     = O.orgid )
```

Week 8 – SQL and Modelling (Part 4): M:M, ERD design method, Dates, Subqueries, Outer Joins, Set Operators, Constraints

8.1 Many-to-many relationships (Lecture 8 Part 1)

- A M:M relationship is perfectly valid in an **ERD**, but the **Relational Model cannot directly represent a M:M** – it must be converted into **two M:1 relationships**.
- **Conversion procedure:** erase the M:M; draw a **weak entity** with an appropriate name in its place; draw **two M:1 relationships**; the MANY end of each attaches to the weak entity; **underline the relationship names**.
- Example STUDENT – SUBJECT: ENROLMENT has no attributes of its own; it is a weak entity so part of the identifier is borrowed; "makes" is underlined → borrow StuNo; "for a" is underlined → borrow SubCode. **Identifier of ENROLMENT = StuNo + SubCode.**
- **No restriction** on subjects-per-student or students-per-subject; adding more subjects/students changes no table design. The weak entity can carry extra attributes (enroldate, finalgrade) – then identifier testing matters (e.g. allow re-enrolment: add enroldate).
- The new weak entity is called an **Intersection Entity** – "has no additional attributes" and "is the result of expanding a M:M relationship (or multi-valued attribute)".

	Unexpanded M:M	Expanded M:M
Advantages	Weak intersection entity hidden → fewer entities and relationships; less cluttered, easier to read; relationship names carry more useful descriptions; non-database people understand it after 30–60 seconds of tuition about symbols	Shows every table that will be generated; nothing hidden; direct conversion to a relational schema
Disadvantages	Harder to determine how many tables will be generated; weak entity is hidden so detail is lost	Cluttered; symbols more confusing; non-database people struggle ("What is the difference between a student and a student society member? Why are they two entities?")

Unexpanded ERDs are sometimes drawn as **documentation** for non-IT readers; expanded ERDs are needed to design the database (and to count tables).

8.2 Designing your own ERD – the nine steps

Step	Action
1	Identify all **strong entities** and all **attributes**
2	Assign attributes to the strong entities
3	Identify **relationships**
4	Determine **cardinality** and **participation** constraints
5	Attach **remaining attributes** to relationships
6	**Expand M:M** relationships (those with attributes)
7	Attach (re-attach) attributes to the appropriate entities or relationships
8	Determine **identifiers**
9	**Test** your solution: create test data; read the ERD aloud; navigate the diagram; think how SQL queries would be written once transformed into a database

- **Identifying entities:** each entity must have **many instances** ("The Business" is therefore not an entity); a **singular noun** (not a verb), written in UPPERCASE; use nouns specific to the business (customer vs client vs patron). May be strong (identified by own attributes e.g. StudentId) or weak (identified wholly or partly by borrowed attributes e.g. Enrolment).
- **Identifying attributes:** identified by a single word or phrase (use underscores or capitalisation to separate words); an attribute has **no properties of its own** (if it does it is probably an entity); it is **dependent on the identifier** of its entity (StuAddress depends on StudentId).

- **Relationships** use relevant verbs; must be **worth recording** and **persistent, not transient** ("the student enrolled in a subject" rather than "the student read the course-finder web page"); watch for abstract nouns ("a student makes an enrolment" ≡ "a student enrols in a subject"); you should be able to verbalise it ("A Subject has many Enrolments").

Worked narrative – Glenferrie Toy Library

"Glenferrie Toy Library has many Patrons. A Patron has a name and a gender. Each new Patron is assigned a sequential patron number. The library has a number of toys. Every toy has a unique toy number, a description and year of acquisition. Each time a toy is loaned to a patron, the date of the loan is recorded. When the toy is returned, the returned date is recorded and the no. of days borrowed is determined. Each loan is for a single toy."

Element	Classification
Glenferrie Toy Library	Ignore (the business is not an entity)
Patron	Strong entity
PatronNo	Attribute (identifier of PATRON)
Firstname, Surname, Patron Gender	Attributes of PATRON
Toy	Strong entity
ToyNo, Description, YearAcquired	Attributes of TOY (ToyNo identifier)
DateBorrowed, DateReturned	Attributes that belong to the borrowing relationship (LOAN)
DaysBorrowed	Calculated (derived from the two dates) – not stored

- **Step 1**: strong entities PATRON {Gender, PatronNo, Firstname, Surname} and TOY {Description, YearAcquired, ToyNo}; DateBorrowed and DateReturned unassigned (the step is useful when many terms are foreign/unfamiliar).
- **Step 2**: attach attributes and underline identifiers. A patron has ONE surname and ONE gender. Where does **DateBorrowed** belong? On PATRON it would allow only ONE DateBorrowed per patron – nonsense (a patron borrows many times). On TOY the same problem. So it belongs to neither – leave it for later steps.
- **Step 3**: relationship "A patron borrows toys". **Step 4**: cardinality/participation – a patron borrows many toys and a toy is borrowed by many patrons → **M:M**. **Step 5**: DateBorrowed and DateReturned belong to the relationship. **Step 6**: expand the M:M with attributes into the weak entity LOAN with two M:1 relationships. **Step 7**: re-attach: each LOAN has a single DateBorrowed (yes) and a single DateReturned (yes, may be NULL). Sometimes you discover more underlying M:M relationships, so don't assume both dates belong to LOAN.
- **Step 8: identifier of LOAN?** If unsure, create the relational schema and test with sample data.

Testing identifiers with sample data

```
PATRON (PatronNo, Name, Gender)                PK: PatronNo
TOY    (ToyNo, Description, YearAcquired)      PK: ToyNo
LOAN   (PatronNo, ToyNo, DateBorrowed, DateReturned)
         PK: PatronNo, ToyNo        FK1: PatronNo → PATRON   FK2: ToyNo → TOY    -- first attempt: WRONG
```

PatronNo	ToyNo	DateBorrowed	DateReturned	Verdict with PK = PatronNo+ToyNo
53	27	1 June	8 June	OK
53	31	1 June	8 June	OK
53	27	19 July	30 July	**Duplicate PK (53,27)!** – the schema never lets a patron borrow the same toy twice

- Fix: a patron must be able to borrow the same toy on multiple dates → underline **DateBorrowed**: PK = **PatronNo + ToyNo + DateBorrowed**. Test rows (53,27,1 June), (53,31,1 June), (53,27,19 July), (53,27,2 August – not yet returned), (71,27,11 June), (71,31,11 June), (71,28,31 July), (53,31,9 October – not returned) → all valid.
- **Would DateReturned belong in the identifier?** Three possible answers: should be part / must not be part / makes no difference. Testing: PK = PatronNo, ToyNo, DateBorrowed, DateReturned → the row (53, 27, 2 August, **NULL**) fails because **a primary key must not contain NULLs**. Hence

DateReturned must NOT be part of the identifier.

8.3 Dates, subqueries, outer joins (Lecture 8 Part 2)

> **NOTE – Level note from the lecture**
> The "advanced" SQL (date functions, outer joins, subqueries, unions, constraints) is not required of students hoping only for a **Pass** – they need not attempt these in PASS submissions or in Test 2. Students seeking a higher grade (D / HD, Test 3) should try some or all of them.

Oracle dates

```
CREATE TABLE STUDENT ( stuID NUMBER PRIMARY KEY, birthdate DATE );
INSERT INTO STUDENT (stuid, birthdate) VALUES (100, '21-MAY-1996');                       -- Oracle
    default DD-MON-YYYY
INSERT INTO STUDENT (stuid, birthdate) VALUES (101, TO_DATE('21/05/1996','dd/mm/yyyy'));  -- any other
    format via TO_DATE
SELECT * FROM STUDENT WHERE birthdate = '21-MAY-1996';
SELECT * FROM STUDENT
WHERE  birthdate >= TO_DATE('01/01/1996','dd/mm/yyyy')
AND    birthdate <= TO_DATE('31/12/2016','dd/mm/yyyy');
```

> **NOTE – Slide typo**
> The slide shows an unbalanced bracket (`TO_DATE(...)` in the range query; in a correct statement every TO_DATE(…) is closed. TO_DATE specifies the format of the incoming date.

Subqueries

- Problem: "find all movies longer than the average". `SELECT * FROM MOVIE WHERE runtime > AVG(runtime);` is **incorrect – aggregate functions are NOT permitted in the WHERE clause** (an aggregate must be evaluated in the context of its own query). Solution: a **subquery**.
- A **subquery is a complete SQL statement placed WITHIN an existing statement**. Used when a result cannot be calculated by a single SQL statement (two or more statements needed). Permitted in the **WHERE** clause (and HAVING). Because it is in brackets it is **evaluated first** (like BODMAS) and **executed ONCE**, not for every row. The single value returned is compared with the runtime of each row.

```
-- Example 1: longer than average
SELECT * FROM MOVIE WHERE runtime > ( SELECT AVG(runtime) FROM MOVIE );
-- Example 2: the longest movie (build the inner query first, then the outer)
SELECT * FROM MOVIE WHERE runtime = ( SELECT MAX(runtime) FROM MOVIE );
-- Example 3: movies with Canadian actors (subquery with WHERE ... IN)
SELECT m.title
FROM   movie m INNER JOIN casting c ON m.movieno = c.movieno
WHERE  actorno IN ( SELECT actorno FROM actor WHERE bornin = 'Canada' );
-- Example 4: subquery in HAVING - rating codes whose average runtime exceeds the overall average
SELECT   rating_code, AVG(runtime)
FROM     movie
GROUP BY rating_code
HAVING   AVG(runtime) > ( SELECT AVG(runtime) FROM movie );
```

> **EXAM TRAP – Slide typos / rules**
> Example 4 on the slide ends with a stray semicolon inside the bracket (`SELECT AVG(runtime) FROM movie;` – the subquery must be closed with a bracket and no semicolon. Use **=** (or >, < …) only when the subquery returns **one value**; use **IN / NOT IN** when it can return many. The slide's "bornin" column is called BirthCountry in the lab movie database.

Join types – inner and outer

- Given Table 1 and Table 2 with a PK:FK relationship: **INNER JOIN** uses values that have a match in both tables. Business requirement "list all actors and the movies each appears in": the inner join of ACTOR and CASTING **loses Brad Pitt** because he has no CASTING row → requirement not met.

- **LEFT OUTER JOIN**: all rows of the LEFT table (first table listed) plus the matching rows of the right. **RIGHT OUTER JOIN**: all rows of the RIGHT table (second table listed) plus matches. Unmatched rows show NULL for the other table's columns. LEFT vs RIGHT depends only on the **order of the table names** – swapping the order and the keyword gives an identical result.

```
SELECT a.fullname, c.movieno
FROM   actor a LEFT OUTER JOIN casting c ON c.actorno = a.actorno
ORDER BY a.fullname;                       -- Brad Pitt listed with a NULL movieno

SELECT a.fullname, c.movieno
FROM   casting c RIGHT OUTER JOIN actor a ON c.actorno = a.actorno
ORDER BY a.fullname;                       -- identical result

-- Inner + outer combined: INNER JOIN casting-movie first, then RIGHT OUTER JOIN actor
SELECT a.fullname, c.movieno, m.title
FROM   casting c
INNER JOIN movie m ON c.movieno = m.movieno
RIGHT OUTER JOIN actor a ON c.actorno = a.actorno
ORDER BY a.actorno;
```

Fullname	MovieNo	Title (lecture result set)
Amy Adams	721	La La Land
Brad Pitt	(NULL)	(NULL)
Natalie Portman	731	Black Swan
Natalie Portman	711	V for Vendetta
Robert Downey Jr	745	Iron Man

8.4 Set operators, literals, constraints (Lecture 8 Part 3)

- **UNION** combines the results of two queries into a single result set: all rows selected by either query, **duplicates removed**. **UNION ALL**: all rows from either query **including duplicates**. The **number and data types** of the selected columns must be the same in each component query (column lengths may differ).
- STUDENT {Amanda Allen, Bob Black, Carol Chen, Dave Dole}, TUTOR {Carol Chen, Ed Ellis, Fred Foley}: UNION → 6 names (Carol once); UNION ALL → 7 rows (Carol twice).
- **Column types must match**: length and precision may vary; CHAR and VARCHAR may be mixed; use conversion functions **TO_CHAR()**, **TO_NUMBER(string [, format_mask])** e.g. TO_NUMBER(amt,'999.99'), TO_CHAR(emp_postcode), or CAST()/CONVERT() (non-Oracle).
- **INTERSECT**: all **unique** rows selected by **both** queries (Carol Chen). **MINUS**: unique rows selected by the **first query but not the second** (STUDENT MINUS TUTOR = Amanda Allen, Bob Black, Dave Dole; **TUTOR MINUS STUDENT = Ed Ellis, Fred Foley** – swapping the queries changes the result).
- **Literals**: a literal value is "hard-coded" into the query, not generated by a value in the table; useful when columns do not match between tables. `SELECT name, 'Unknown' FROM customer WHERE gender IS NULL` gives Chris Unknown / Lee Unknown / Taylor Unknown; combine with UNION to list all customers.

```
SELECT name FROM STUDENT UNION     SELECT name FROM TUTOR;
SELECT name FROM STUDENT UNION ALL SELECT name FROM TUTOR;
SELECT name FROM STUDENT INTERSECT SELECT name FROM TUTOR;
SELECT name FROM STUDENT MINUS     SELECT name FROM TUTOR;

SELECT custnumb, 'Good Customer' AS Rating FROM customer WHERE balance <= credlim
UNION
SELECT custnumb, 'Bad Customer'  AS Rating FROM customer WHERE balance >  credlim
ORDER BY custnumb;
```

EXAM TRAP – Access and set operators
MS Access supports UNION and UNION ALL but not INTERSECT or MINUS ("property not found"). Emulate them with `WHERE x IN (SELECT …)` and `WHERE x NOT IN (SELECT …)` (or use Oracle iSQL Jr).

Constraints: UNIQUE and CHECK

- Already used: **Primary Key, Foreign Key, Not Null**. A **UNIQUE constraint** ensures a column value is never duplicated: `title VARCHAR(30) UNIQUE`. A unique constraint on more than one column must be a **table constraint**: business rule “an actor name must never be repeated” → , `UNIQUE(surname, firstname)`.
- **CHECK constraint** = fixed validation of data values, e.g. `gender VARCHAR(1) CHECK (gender IN ('M','F','I'))` – Male, Female or Intersex; this complies with Australian Standard AS 4590:2006 “Interchange of client information”; Australian government agencies must offer the intersex option and discrimination on the basis of sexual orientation, gender identity or intersex status is illegal.
- More examples: `test1 NUMBER CHECK (test1 >= 0)`; `test2 NUMBER CHECK (test2 BETWEEN 0 AND 50)`; a table-level check `CHECK (test1 + test2 <= 100)`.
- **Use CHECK only for specific values that are fixed and will never change** (`CHECK (dayname IN ('Mon',…,'Sun'))`). Do not use it when values may change (`managername IN ('Dave','Sue','Henry')`, `grade IN ('Good','Excellent','Poor')`) – adding/deleting values requires modifying a table that may hold many rows; create a **separate look-up table** that the end-user can maintain.
- **Check rules:** a row passes if the condition is **TRUE or NULL (unknown)**. Syntax is the same as WHERE, except a CHECK cannot contain **conditions referring to other rows or other tables, subqueries, or calls to SQL functions such as SYSDATE**. Violation → `ORA-02290: check constraint violated`; duplicate on UNIQUE/PK → `ORA-00001: unique constraint violated`.

Appendix A – Access vs Oracle SQL: side-by-side

Topic	Oracle (iSQL*Plus Junior, used in the unit)	MS Access SQL / Query design
Text literals	Single quotes: 'M'	Double quotes: "M"
Case sensitivity of text	Case SENSITIVE ('Smith' ≠ 'SMITH')	NOT case sensitive
Wildcards	LIKE with % (many chars) and _ (one char)	LIKE with * (many), ? (one), # (digit), [ ] lists
Upper / lower	UPPER(), LOWER()	UCase(), LCase() (UPPER is “Undefined function”). Power BI uses upper()/lower()
Substring / length / concatenation	SUBSTR(), LENGTH(), ||	Mid(), Left(), Right(), Len(), &
Dates	'21-MAY-1996' or TO_DATE('21/05/1996','dd/mm/yyyy'); TO_CHAR for formatting	#1996-05-21# literal; Date(), Now(), DateValue(), Format()
Data types	VARCHAR(n), NUMBER(p,s), DATE	Short Text, Number, Currency, Date/Time, AutoNumber
Joins with 3+ tables	Plain INNER JOIN … ON … INNER JOIN … ON …	Each join must be wrapped in brackets: FROM ((A INNER JOIN B ON …) INNER JOIN C ON …)
Set operators	UNION, UNION ALL, INTERSECT, MINUS	UNION, UNION ALL only (INTERSECT / MINUS → use IN / NOT IN)
Statements	Many statements in a script separated by ; (iSQL Jr executes one at a time)	One statement per query; no ; in the middle
Column aliases	AS "My Heading" (double quotes)	AS MyHeading or Heading: field in the design grid
Constraints	CREATE TABLE … PRIMARY KEY / FOREIGN KEY / NOT NULL / UNIQUE / CHECK	Design view: Required, Indexed (No duplicates), Validation Rule (CHECK equivalent), Relationships window
Transactions	COMMIT / ROLLBACK (autocommit behaviour of DDL)	Action queries cannot be undone
Errors to know	ORA-00001 unique, ORA-00911 invalid character (smart quotes), ORA-00937 not a single-group group function, ORA-01400 cannot insert NULL, ORA-02290 check violated, ORA-02291 parent key not found, ORA-02292 child record found	Duplicate PK message; referential integrity violation; “Syntax error in FROM clause” (missing brackets); “Undefined function”

Appendix B – Laboratory tasks 1–5: what each task practises

Task	Pass level	Credit level (and above)
Task 1 – Filters, Queries, Relations, Functions, Forms (Access)	1a Filters vs queries (advantages of queries); 1b single query on movies of 2012; 1c/1d queries with multiple criteria (AND/OR/LIKE/range) – the grid must show ALL criteria; 1e create Member table with 5 rows; 1f Relationships diagram (all table names carry your 4-digit ID); 1g query over MOVIE, RATING, COLOURTYPE; 1h totals: actor no, name, Count of CastId; 1i rows per gender; 1j Index on Surname (duplicates OK); 1k …	1a–1d queries on cust.accdb with several AND/OR criteria (state, gender, loyalty points range, rating 1 or 3) and Date-range criteria; 1d add yourself as a customer; 1e–1f product_sales.accdb (350,000+ sales): relationships on POSTCODE, PRODID, PRODTYPE, one query with joined columns and multi-criteria
Task 2 – Exchange and Visualisation	2a import studentlist.csv (edit in Notepad++, not Excel; date format!); 2b questions on import/link; 2c add field, sort, copy table to STUCOPY9999; 2d Update query (FeesPaid = 4000 for males); 2e Delete query (surname begins with M); 2f parameter query (PostCode); 2g Power BI Get Data from the movie database, relationships; 2h Table visual, Matrix, slicers …	2a import Excel (2 worksheets) + relationships; 2b DateDiff() age query; 2c parameter query on age; 2d Totals query (count by suburb/type/description); 2e export query to text file (semicolon delimiter, single-quote qualifier); 2f–2h Power BI: new column TotalPets = NoOfDogs + NoOfCats, Weekday() column, donut chart, titles with student ID
Task 3 – SQL and Schemas (Oracle iSQL Jr)	3a run moviedb_sql script with your suffix (movie9999); 3b–3i SELECT, column lists, ORDER BY, IN/BETWEEN/LIKE, AND/OR with parentheses, 2-table inner join; follow the CODES AND STANDARDS (keywords uppercase, one clause per line)	3a–3b multi-criteria and rating/runtime combinations with a join; 3c DISTINCT release years descending; 3d UPDATE tmdb_votes then SELECT; 3e NOT LIKE on letters A B I O; 3f–3i design CAR9999 schema, DROP/CREATE TABLE, INSERT script, SELECT with conditions
Task 4 – ERDs, Joins and Clauses	4a ERD → relational schema (PLAYER–CAMPAIGN); 4b CREATE TABLEs with PK/FK (suffix in names, no surrogate keys); 4c INSERTs; 4d DESCRIBE + SELECT *; 4e deliberately FAIL: duplicate PK, FK parent missing (ORA-02291), delete parent with child (ORA-02292) – and explain; 4f join; 4g COUNT(*), COUNT by campaign; 4h/4i Access SQL on actor/movie; 4j SwinEats ERD (M:M with date and fee → weak entity); 4k Hawthorn Comedy ERD (rating of shows by attendees → weak entity with a score)	4a 2 inner joins (movie, colour name, rating description) in descending title order; 4b actor name + title + year per casting row; 4c GROUP BY rating code count (descending count); 4d HAVING count > 21; 4e WHERE years 1996–2010; 4f actor + gender + number of movies (alias “Total Movies”); 4g actor + rating code + count; 4h SUM of sales this year by state/gender with rating code 1,2,3 plus a written comment
Task 5 – ERDs and Normalisation	5a Pete’s Pools ERD (clients join centres, date joined → weak entity, no surrogate keys); 5b schema; 5c CREATE TABLE; 5d explain normalisation; 5e unnormalised car-rental table → 1NF	5a re-joining allowed (identifier must include the join date); 5b Tiny’s Motorcycle ERD with surrogate keys where required; 5c schema; 5d CREATE TABLE; 5e 3NF; 5f transactions: purchase/return SQL, COMMIT, ROLLBACK
Distinction / HD	Sample Answer provided in the repository (Laboratory/Distinction Task)	

Appendix C – Normalisation and transactions (Task 5 background, Lectures 9–11 preview)

- **Normalisation** = the process of organising data into well-structured tables to reduce redundancy and avoid **update, insert and delete anomalies**, so each fact is stored once.
- **Unnormalised** (UNF): repeating groups in a row (a customer with several cars and rental dates in one row). **1NF**: every cell holds a single value, no repeating groups; each row identified by a key (e.g. CustId + CarRego + StartDate). **2NF**: 1NF and no partial dependency (non-key attributes depend on the whole composite key). **3NF**: 2NF and no transitive dependency (non-key attributes depend only on the key, “the key, the whole key and nothing but the key”). Example 3NF split: CUSTOMER(CustId, Name, Phone), CAR(CarRego, MakeModel), RENTAL(CustId, CarRego, StartDate, ReturnDate).
- **Transaction** = a logical unit of work (e.g. a purchase: INSERT into ACTION and UPDATE of PRODUCT.QtyInStock) that must either complete fully or not at all. **COMMIT** makes all changes since the last commit permanent; **ROLLBACK** undoes all uncommitted changes. A sale can go wrong if stock is insufficient or the system fails between the two statements – then ROLLBACK restores consistency. **ACID**: Atomicity, Consistency, Isolation, Durability.

Appendix D – SQL cheat sheet synthesis (Week 5 cheat sheets 01 and 02)

Group	Command	Syntax / example
DML	SELECT / INSERT / UPDATE / DELETE	SELECT c1,c2 FROM t; INSERT INTO t (c1,c2) VALUES (v1,v2); UPDATE t SET c1=v1 WHERE cond; DELETE FROM t WHERE cond;
DDL	CREATE / ALTER / DROP / TRUNCATE	CREATE TABLE t (c1 INT PRIMARY KEY, c2 VARCHAR(50)); ALTER TABLE t ADD email VARCHAR(100); DROP TABLE t; TRUNCATE TABLE t; (removes all rows, keeps the table)
DCL	GRANT / REVOKE	GRANT SELECT, INSERT ON employees TO user; REVOKE SELECT, INSERT ON employees FROM user;
Query clauses	WHERE / ORDER BY / GROUP BY / HAVING	SELECT category, COUNT(*) FROM products GROUP BY category HAVING COUNT(*) > 5;
Joins	INNER, LEFT [OUTER], RIGHT [OUTER], FULL [OUTER], CROSS, SELF, NATURAL	FULL JOIN = all rows when there is a match in either table (Oracle supports FULL OUTER JOIN; the sheet also shows LEFT JOIN UNION RIGHT JOIN); CROSS JOIN = Cartesian product; SELF JOIN joins a table with itself using two aliases; NATURAL JOIN matches same-named columns
Subquery operators	IN / ANY / ALL	WHERE city IN (SELECT city FROM suppliers); price < ANY (subquery); order_amount > ALL (subquery)
Aggregates	COUNT, SUM, AVG, MIN, MAX	COUNT(column) ignores NULLs; COUNT(*) counts rows
String functions	CONCAT, SUBSTR/SUBSTRING, LENGTH, UPPER, LOWER, TRIM, LEFT/RIGHT, REPLACE	Oracle: SUBSTR(s,start,len), LENGTH(s), s1 || s2, TRIM, REPLACE(s,old,new)
Date functions	CURRENT_DATE, EXTRACT, TO_CHAR, DATEDIFF …	Oracle: SYSDATE, TO_CHAR(d,'YYYY-MM-DD'), EXTRACT(YEAR FROM d), ADD_MONTHS; MySQL: DATE_ADD, DATEDIFF
Conditional	CASE, IF, COALESCE, NULLIF	CASE WHEN total > 1000 THEN 'High' WHEN total > 500 THEN 'Medium' ELSE 'Low' END AS status; COALESCE(a,b) = first non-NULL; NULLIF(a,b) = NULL if equal
Set operations	UNION, INTERSECT, EXCEPT (MINUS in Oracle)	SELECT … UNION SELECT …;
Transactions	COMMIT / ROLLBACK / SAVEPOINT / ROLLBACK TO / SET TRANSACTION	SAVEPOINT s1; … ROLLBACK TO s1; COMMIT; SET TRANSACTION ISOLATION LEVEL READ COMMITTED | SERIALIZABLE

Cheat sheet 02 (worked tutorial with employees, departments, products and orders tables) repeats these ideas with sample data: SELECT with expressions/aliases, WHERE with BETWEEN / LIKE / IS

NULL, ORDER BY, GROUP BY with aggregates, UPDATE and DELETE examples, ALTER TABLE, and INNER / LEFT / RIGHT / FULL / CROSS / SELF joins plus a join with GROUP BY (total sales per product category).

Appendix E – Tutorial question coverage (Weeks 1–8)

Week	Tutorial themes (what could be asked again)
1	Structured/semi/unstructured data; RDBMS vs spreadsheet; GUI vs SQL; Access views; record/field vs row/column; primary key & AutoNumber; filter vs query; AND/OR criteria rows; wildcards
2	Why separate tables; FK/PK and Many-to-One; referential integrity; Cartesian product; totals queries; indexes (advantages/disadvantages, binary search)
3	Parameter queries; make-table/append/update/delete (no undo); crosstab; import vs link; CSV/fixed-width/XML/Excel dates
5	ARD vs ERD; natural vs surrogate keys; business rules; reading cardinality; relational schema with FK at the MANY end; CREATE TABLE and INSERT; spot invalid SELECTs (missing FROM, unquoted literals, IS vs =); what CREATE TABLE does when the table exists
6	Foreign keys in a schema; NULL in FK; table creation order (parents first: LEADER → PROJECT → TASK); parent/child true-false; ORA-02292; row-count questions with AND/OR/NOT/LIKE (e.g. NOT (QtyA>=5 AND QtyB<5) ignores NULLs); inner joins with 2 and 3 tables
7	COUNT(*) outputs; GROUP BY legality (ORA-00937); multi-column GROUP BY; HAVING; MIN; composite identifiers: ALLOCATION(EmpNo, TaskNo), ATTACHED(WID, JobName, AgreedRate, DateNegotiated), HOUSE(SuburbName, StreetName, HouseNo, …)
8	Narrative → ERD → schema: GlenCo (employee–equipment 1:M), Eastern Primary mascot vote (M:M → VOTE weak entity), Sudoku championship (M:M with time), Swinny Publishing (1 author → many authors → bookshop orders → orders with many books)